

Expresiones Lambda e Interfaces Funcionales

Expresiones lambda

Una de las nuevas utilidades introducidas en Java 8 fueron las expresiones lambda. Imaginemos que tenemos un interfaz muy simple, con un método abstracto (recordemos que todos los métodos de interfaz son abstractos a menos que estén declarados default o static):

```
interface PaseadorPerros{  
    public boolean pasear(Perro p);  
}
```

PaseadorPerros es **un interfaz funcional: un interfaz con un método abstracto**. Ahora usamos una clase interna para definir una instancia de una clase que implementa a este interfaz:

```
PaseadorPerros paseador=new PaseadorPerros(){  
    public boolean pasear(Perro p){return p.getEdad(>9;}  
};
```

Podemos hacer este código mucho más conciso reemplazando la clase interna por una expresión lambda:

```
PaseadorPerros paseador= (d)->d.getEdad(>9;
```

Usaremos esta expresión lambda como usaríamos una instancia de una clase interna.

Pensemos en una expresión lambda como un atajo para escribir una instancia de una clase que implementa un interfaz funcional. Para hacer la expresión lambda, copiamos el parámetro del método test() desde la clase interna, después escribimos una flecha, y después copiamos la expresión en el cuerpo del método test, sin poner el return ni el punto y coma.

```
public boolean test(Dog d) { return d.getAge() > 9; }  
                                (d) -> d.getAge() > 9
```

El tipo de la expresión lambda es PaseadorPerros. El mismo que le tipo del interfaz y el mismo que el tipo de la instancia que está siendo creado por la clase interna. Como el método test() tiene un único parámetro, es perfectamente legal eliminar los paréntesis alrededor del parámetro:

```
PaseadorPerros paseador= (d)->d.getEdad(>9;
```

Si tenemos más de un parámetro, es obligatorio usar paréntesis. Por otra parte nos podemos preguntar: ¿Cómo sabe la lambda cuál es el tipo de **d**? Este puede ser inferido de la definición en el interfaz. Puede haber veces en que no se pueda inferir y necesitaremos escribirlo:

```
PaseadorPerros paseador= (Perro d)->d.getEdad(>9);
```

¿Y el tipo de retorno? También puede ser definido de la definición del interfaz. La regla es que **si sólo hay una expresión en la lambda, entonces el valor de la expresión es retornado por defecto**, y no necesitamos retornarlo. Si queremos escribirlo, la sintaxis es:

```
PaseadorPerros paseador= (d)->( return d.getEdad(>9);
```

Si el cuerpo de la lambda tiene más de una expresión (una o múltiples sentencias), necesitaremos usar llaves:

```
PaseadorPerros paseador = d -> {  
    System.out.println("Nombre del perro: " + d.getNombre( ));  
    return d.getEdad( ) > 9 ;  
};
```

Para resumir, podemos escribir la instancia original de PerroQuerier de las siguientes maneras:

```
PaseadorPerros paseador = (d) -> d.getEdad( ) > 9;  
PaseadorPerros paseador = d -> d. getEdad ( ) > 9;  
PaseadorPerros paseador = (Perro d) -> d.get getEdad ( ) > 9;  
PaseadorPerros paseador = d -> { return d. getEdad( ) > 9; };
```

A continuación un ejemplo completo:

```
public class Perro {  
    private String nombre;  
    private int peso;  
    private int edad;  
    public Perro(String nombre, int peso, int edad) {  
        this.nombre = nombre;  
        this.peso = peso;  
        this.edad = edad;  
    }  
    public String getNombre() {  
        return nombre;  
    }  
    public int getPeso() {  
        return peso;  
    }  
    public int getEdad() {  
        return edad;  
    }  
    @Override  
    public String toString() {  
        return "Perro{" + "nombre=" + nombre + ", peso=" + peso + ", edad=" + edad + '}';  
    }  
}
```

```

interface PaseadorPerros{
    public boolean probar(Perro p);
}
public class TestPerros {
    public static void main(String[] args){
        Perro perro1 = new Perro("Sultan",30,6);
        Perro perro2 = new Perro ("Canelo",35,12);
        /* PaseadorPerros dq = new PaseadorPerros( ) {
            public boolean probar(Perro d) { return d.getEdad( ) > 9; }
        }; */
        PaseadorPerros dq = d -> d.getEdad( ) > 9; // Reemplaza a la clase interna.
        System.out.println("¿Es Sultán mayor de 9 años? " + dq.probar(perro1));
        System.out.println("¿Es Canelo mayor de 9 años? " + dq.probar(perro2));
    }
}

```

Algunos ejemplos de expresiones lambda no válidas:

```

Oveja s -> s.color.equals(color); // Necesita paréntesis alrededor del argumento.
(Oveja s) -> if (s.color.equals(color)) return true; else return false; //Necesita llaves
// alrededor de la lambda.
PaseadorOvejas tetsOveja = s, n -> s.edad > n; // Necesita paréntesis alrededor de los
// argumentos.

```

Pasando expresiones lambda a métodos

Añadamos una clase a nuestro ejemplo anterior:

```

class GestionPerros {
    PaseadorPerros paseador;
    public GestionPerros(PaseadorPerros paseador ) {
        this.paseador = paseador;
    }
    public boolean darPaseo(Perro d) {
        return paseador.probar(d); //probar() es abstracto. Está definido en el interfaz.
    }
}

```

El constructor de la clase GestionPerros toma una instancia de PaseadorPerros. Podemos pasar la instancia dq que creamos con una expresión lambda:

```
GestionPerros dp = new GestionPerros (dq);
```

O podemos pasar la expresión lambda directamente:

```
GestionPerros dp = new GestionPerros (d -> d.getEdad( ) > 9);
```

Cuando llamamos a dp.darPaseo() y pasamos un Perro, el método probar() de PaseadorPerros es invocado:

```
System.out.println("Es Sultan mayor de 9 años? " + dp.probar(perro1));
```

Accediendo a variables desde expresiones lambda

¿Qué ocurre si una lambda tiene una referencia a otra variable? Mientras podemos declarar y usar variables dentro de la expresión lambda, igual que lo haríamos en el cuerpo de un método, la lambda esta esencialmente creando un bloque anidado, de forma que NO podemos usar el mismo nombre de variable que se ha usado en el alcance del contenedor de la lambda. Esto es correcto:

```
int numPerros = 3;
PaseadorPerros paseador = d -> {
    int numCollares = 1 ;    // Variable local completamente nueva.
    numCollares ++;          // Podemos modificar numBalls.
    System.out.println("Número de collares: " + numCollares);
    System.out.println("Número de perros: " + numPerros);
    return d.getEdad( ) > 9;
};
```

Pero esto **NO** lo es:

```
int numPerros = 3;
int numCollares = 1; // Ahora tenemos a numBalls en el alcance del contenedor.
PaseadorPerros paseador = d -> {
    int numCollares = 1 ;    // No compilará. Tratamos de redeclarar numCollares.
    System.out.println("Número de collares: " + numCollares);
    System.out.println("Número de perros: " + numPerros);
    return d.getEdad( ) > 9;
};
```

Una expresión lambda "captura" variables del alcance del contenedor, por lo que se puede acceder a esas variables en el cuerpo de la lambda, pero esas variables debe ser **final** o **efectivamente final**. Una **variable final efectiva** es una variable o parámetro cuyo valor no cambia después de que se inicializa. Veamos qué ocurre si tratamos de modificar el número de cats en la expresión lambda:

```
int numPerros = 3;
PaseadorPerros paseador = d -> {
    int numCollares = 1 ;
    numCollares ++;
    numPerros++; // No compilará, no podemos cambiar numCats
    System.out.println("Número de collares: " + numCollares);
    System.out.println("Número de perros: " + numPerros);
    return d.getEdad( ) > 9;
};
```

Podemos usar el valor de numCats, pero NO podemos cambiarlo. Lo siguiente funciona:

```
int numPerros = 3; // numCats es efectivamente final
PaseadorPerros paseador = d -> {
    int numCollares = 1;
    numCollares ++;
    System.out.println("Número de collares: " + numCollares);
};
```

```
System.out.println("Número de perros: " + numPerros); // OK usar numPerros
return d.getEdad( ) > 9; };
```

El valor de la variable numPerros es capturado por la lambda de forma que puede ser usado más tarde cuando invocamos a la lambda (al llamar al método probar()). Veamos que ocurre cuando pasamos una lambda con valores capturados al constructor GestionPerros:

```
System.out.println("--- usar GestionPerros ---");
GestionPerros dp = new GestionPerros (paseador);
System.out.println("¿Es Sultán mayor de 9 años? " + dp. darPaseo (perro1));
```

Esto funciona bien; el valor capturado para numPerros es enviado junto con la lambda a GestionPerros y usado cuando la lambda es invocada después cuando invocamos a darPaseo.

Interfaces funcionales

A los interfaces con un y sólo un método abstracto los llamamos interfaces funcionales. Podemos identificar explícitamente a un interfaz funcional usando la notación:

```
@Functional Interface
interface PaseadorPerros{
    public boolean probar(Perro p);
}
```

Esta notación no es obligatoria, pero puede ser útil cuando se desea garantizar que no se agregan accidentalmente métodos a una interfaz funcional que luego romperá otro código. Si usamos el tipo PaseadorPerros para definir una lambda expresión y luego agregamos otro método abstracto a esta interfaz, sin la anotación **@FunctionalInterface**, obtendrá un **error** del compilador. Con la notación, el compilador nos avisará.

Interfaces funcionales incorporadas

Con **Java 8** nosotros obtenemos una gran colección de interfaces funcionales en el paquete **java.util.function**, donde encontraremos **Predicate**, junto con una variedad de otros.

- ¿Qué hace que un interfaz seas funcional?

La clase **java.util.function.Predicate** tiene cinco métodos: **and()**, **isEqual ()**, **negate()**, **or()** y **test()**. Todos los métodos excepto test() se declaran como static o default, por lo que es bastante fácil determinar que test() es el único método abstracto en esta interfaz funcional. Claramente, **Predicate** es una interfaz funcional.

Ahora, echemos un vistazo a **java.util.Comparator**. Dos métodos: **compare()** y **equals()**. Pero dijimos que esto es un interfaz funcional, entonces, ¿por qué se enumeran aquí dos métodos abstractos? **equals()** se hereda de **Object** y los métodos públicos heredados no se cuentan cuando está determinando si una interfaz es una interfaz funcional. Aunque **equals()** sea abstracto en **Comparator** no cuenta debido a que es heredado.

Una interfaz funcional es una interfaz que tiene un método abstracto (método funcional). Los métodos default no cuentan; los métodos estáticos no cuentan; y los métodos heredados de **Object** no cuentan.

Categorías de interfaces funcionales

Los tipos básicos de interfaces funcionales son 4: **Proveedores, Consumidores, Predicados y Funciones**.

- **Proveedores (Suppliers)** Proporcionan resultados. El método **get()** de **java.util.function.Supplier** nunca toma un argumento y siempre retorna algo.
- **Consumidores(Consumers)** Consumen valores. El método **accept()** de **java.util.function.Consumer** siempre toma un argumento y nunca retorna nada.
- **Predicados(Predicates)** Comprueban cosas (y hacen operaciones lógicas). El método **test()** de **java.util.function.Predicate** toma un valor, hace un test lógico y devuelve verdadero o falso.
- **Funciones(Functions)** Es el interfaz funcional más genérico. El método **apply()** de **java.util.function.Function** toma un argumento y retorna un valor.

Todas las interfaces funcionales en **java.util.function** son variaciones de proveedores, consumidores, predicados y funciones.

Trabajando con Proveedores

Comencemos con un proveedor muy sencillo:

```
Supplier<Integer> proveedor = ( ) -> 42;  
System.out.println("Responde a todo: " + proveedor.get());
```

Este es un **Supplier** cuyo método funcional **get()** devuelve un objeto Integer, 42. Mirando a su definición en **java.util.function.Supplier**:

```
@FunctionalInterface  
public interface Supplier<T> // T indica el tipo que será retornado por el método funcional.
```

Veamos cómo podríamos crear un Supplier sin usar una expresión lambda, con una clase interna:

```
Supplier<Integer> proveedorInnerClass = new Supplier<Integer>( ){  
    public Integer get( ) {  
        return 42;  
    }  
};
```

Probemos otro ejemplo:

```
Supplier<String> userSupplier = ( ) -> {  
    Map<String, String> env = System.getenv( ); // Toma el mapa del entorno del  
                                                // sistema.  
    return env.get("USER"); // Toma el valor con la clave "USER" del mapa y lo  
                           // devuelve. En sistemas Windows se usa "USERNAME".  
};  
System.out.println("El usuario es: " + userSupplier.get( ));
```

La clave a tener en cuenta sobre estos dos proveedores es que no aceptan argumentos y devuelven un objeto.

- **IntSupplier**

Mirando **java.util.function**, podemos ver que hay algunas variaciones en el proveedor, incluyendo **IntSupplier**, **DoubleSupplier** y **LongSupplier**. Estos proporcionan un int, un double y un long, respectivamente. Estos están principalmente para evitar el autoboxing en caso de que desee una primitiva, en lugar de un objeto, de regreso del proveedor. Por ejemplo, el método funcional de **IntSupplier** es **getAsInt()**:

```
Random random = new Random( );  
IntSupplier randomIntSupplier = ( ) -> random.nextInt(50);  
int myRandom = randomIntSupplier.getAsInt( );  
System.out.println("Número aleatorio: " + myRandom);
```

Hay que tener en cuenta que **IntSupplier** no utiliza un parámetro de tipo, porque el método funcional devuelve una primitiva. Esto evita el autoboxing a **Integer** que obtenemos con el **Supplier<Integer>**.

- **¿Cuál es el sentido de Supplier?**

Tomemos como ejemplo la clase **java.util.logging.Logger**, que ha sido aumentada con muchos métodos que toman un **Supplier** como argumento. Esta clase es usada para registrar mensajes para un sistema o componente de aplicación y se puede establecer el nivel de registro para determinar qué tipos de mensajes y cuántos mensajes registrar. Podemos usar el método **log()** para registrar una cadena de texto si el nivel de registro está establecido:

```
Logger logger = Logger.getLogger("Registrador de estado.");  
Logger.setLevel(Level.SEVERE);  
  
// Más tarde ...  
String currentStatus = "Todo va bien";  
Logger.log(Level.INFO, currentStatus);
```

En este caso no vemos la cadena del estado actual registrada debido a que el nivel de registro está establecido a **SEVERE** y nuestra llamada al método **log()** dice que sólo se registrarán mensajes si el nivel está establecido en **INFO** o inferior (los niveles están ordenados con **SEVERE** en el nivel más alto e **INFO** un par de niveles por debajo).

```
String currentStatus = "Algo ha ido muy mal";  
Logger.log(Level.SEVERE, currentStatus);
```

Este mensaje sí que será visualizado. Si seguimos adelante y calculamos el estado antes de enviar una cadena al método `log( )`, entonces potencialmente hemos perdido el tiempo revisando el estado del nivel de registro que no resulta lo suficientemente alto como para que realmente tengamos que registrarlo. Aquí es donde el Proveedor resultará útil.

Supongamos que el estado se determina haciendo una llamada a través de la red y comprobando si el sistema está caído o no. Un nuevo método `log( )` en **Logger** toma un `level` y un **Supplier<String>** en vez de un **String**.

Queremos registrar el estado, cuando las cosas están bien, pero solo si el nivel de registro está establecido en INFO o menor.

Por otra parte, no queremos molestarnos en verificar el estado y registrar cualquier cosa si el nivel de registro establecido es SEVERE. En ese caso, solo queremos registrar mensajes que indiquen que las cosas están ¡Muy mal!

Y queremos registrar el estado si las cosas van mal, más o menos no importa cuál es el nivel de registro, así que registraremos el estado si el nivel está establecido a SEVERE o menor (y como SEVERE es el nivel más alto, entonces siempre registramos el estado cuando las cosas van mal).

```
String host = "coderanch.com";
int port = 80;
Logger logger = Logger.getLogger("Status Logger");
Logger.setLevel(Level.SEVERE);
    // En caso de que necesitemos chequear el estado
Supplier<String> status = ( ) -> {
int timeout = 1000;
try( Socket socket = new Socket( )) {
    socket.connect(new InetSocketAddress(host,port),timeout);
    return "up";
}catch (IOException e){ return "down"; } // Error, no puedo alcanzar el sistema
};
try{
    logger.log(Level.INFO,status); // Sólo llama al método get() del Supplier si el nivel
                                // establecido es INFO o inferior
    // Hacer cosas con coderanch.com
}catch (Exception e){
    logger.log(Level.SEVERE, status); // Llama al método get( ) del Supplier si el nivel
                                // es SEVERE o inferior
}
```

El método `log( )` toma al Proveedor y comprueba el nivel del log. Debido a que hemos establecido el nivel de registro en SEVERE, en el bloque `try`, donde invocamos al método `log( )` con `Level.INFO`, el método `log( )` no se molestará en llamar al método `get( )` del Proveedor (su método funcional), y así evitamos hacer la costosa llamada de red.

Entonces, usar un proveedor aquí evita esa operación costosa cuando es innecesario. Estamos pasando un bloque de código (el Proveedor) que se ejecuta solo si se aplica una determinada condición. No tenemos que verificar esa condición Nosotros mismos; Logger lo hace por

nosotros y luego puede llamar al método `get()` (su método funcional) en `Proveedor` solo si es necesario.

Trabajando con Consumidores

Los consumidores aceptan uno o más argumentos y no retornan nada:

```
Consumer<String> rojaOazul = pildora -> {
    if (pildora.equals("roja")) {
        System.out.println("Enter the matrix");
    } else if (pildora.equals("azul")) {
        System.out.println("Stay in lala land");
    }
};
```

El interfaz `java.util.function.Consumer` está definido de la siguiente forma:

```
@FunctionalInterface
public interface Consumer<T>
```

El método funcional de `Consumer` es `accept()`, el cuál toma un objeto del tipo `T` y no retorna nada. Hay variaciones en los consumidores en el paquete `java.util.function`: `IntConsumer`, `DoubleConsumer` y `LongConsumer`, que toman un argumento primitivo. Además están `ObjIntConsumer`, `ObjDoubleConsumer`, `ObjLongConsumer` cuyo método `accept()` toma un objeto (tipo `T`) y un `int`, `double` o un `long`. Finalmente tenemos a **`BiConsumer`**. Su método `accept()` toma dos objetos (que no tienen por qué ser del mismo tipo):

```
@FunctionalInterface
public interface BiConsumer<T,U>
```

con un método funcional: `void accept(T t, U u)`

```
Map<String, String> env = System.getenv();
BiConsumer<String,String> printEnv = (key, value) -> { System.out.println(key + ": " + value);};
printEnv.accept("USERNAME",env.get("USERNAME"));
```

- **ForEach**

El método `forEach()` se ha añadido al interfaz `Iterable` en Java 8. Funciona muy bien con un `Consumer`.

```
List<String> nombresPerro = Arrays.asList("Sultán","Zaida","Canelo");
Consumer<String> imprimirNombre = nombre -> System.out.println(nombre);
nombresPerro.forEach(imprimirNombre); // Pasa el consumidor printName a forEach().
env.forEach(printEnv); // El método ForEach() de Map espera un BiConsumer.
```

Esto muestra todos los pares clave/valor en el `Map` que obtenemos de `System.getenv()`

- **Efectos secundarios desde el interior de lambdas**

Sabemos que si definimos una variable en el entorno que encierra la expresión lambda, no podemos modificar esa variable desde dentro de la lambda:

```
String username;
BiConsumer<String,String> findUsername =
(key, value) -> { if (key.equals("USER")) { username = value; };
// Error al compilar. username debe ser efectivamente final.
env.forEach(findUsername);
```

Sin embargo, aunque no podemos cambiar una variable desde dentro de la lambda **SI podemos modificar un campo en un objeto**. Si queremos usar **forEach()** para iterar a través de una colección de objetos para, por ejemplo, encontrar un valor, nosotros no podemos devolver el valor que encontramos (porque **forEach()** toma un consumidor y el método **accept()** de un consumidor es void), pero podemos cambiar el campo de un objeto desde dentro de la lambda para hacer efectivamente lo mismo:

```
public class Consumers {
    public static void main(String[] args) {
        Map<String, String> env = System.getenv( );
        User user = new User( );
        BiConsumer<String, String> findUserName = (key, value) -> {
            if ( key.equals("USERNAME"))
                user.setUsername(value);
        };
        env.forEach(findUserName);
        System.out.println("Username for env: " + user.getUsername( ));
    }
}

class User {
    String username;
    public void setUsername(String username) {
        this.username = username;
    }
    public String getUsername( ){
        return this.username;
    }
}
```

Sin embargo, este código es más ineficiente que usar la iteración del bucle for.

- **El método andThen()**

Es un método default del interfaz **Consumer** que podemos usar para encadenar consumidores:

```
public class Perro {
    private String nombre;
    private int edad;
    private int peso;
```

```

Perro(String nombre, int peso, int edad)
{
    this.nombre = nombre;
    this.peso = peso;
    this.edad = edad;
    // getters y setters aquí.
    public String getNombre( ) { return this.nombre; }
    public void ladrar( ) { System.out.println("Guau!"); }
}

```

Ahora usamos **forEach()**:

```

public class Consumers {
    public static void main(String[] args) {
        ArrayList<Perro> perros = new ArrayList( );
        Perro perro1 = new Perro("Sultan",30,6);
        Perro perro2 = new Perro("Canelo",35,12);
        Perro perro3 = new Perro("Lucas", 45, 8);
        perros.add(perro1); perros.add(perro2); perros.add(perro3);
        Consumer<Perro> mostrarNombres = d -> System.out.print(d + " ");
        perros.forEach(mostrarNombres);
    }
}

```

Ahora digamos que queremos mostrar el nombre del perro **andThen** (y después) queremos mostrar el ladrar(). Lo podemos hacer con consumidores:

```

perros.forEach(mostrarNombres.andThen(d -> d.ladrar( )));

```

Notemos que el método `forEach( )` está llamando al método `accept( )` del Consumer que le estamos pasando. Así que para cada perro `d` en la lista, `forEach( )` está esencialmente haciendo esto:

```

mostrarNombre.accept(d);

```

Cuando el método **andThen()** del Consumer es invocado, usa el mismo objeto `d` que usamos para el método `accept( )` en el primer Consumer para el método `accept( )` del segundo Consumer. Hemos escrito el segundo Consumer en una lambda de una única línea (en vez de en una declaración separada como hicimos para `mostrarNombres`). Podríamos haber escrito las dos lambdas en una sola línea:

```

perros.forEach((d -> System.out.print(d + " ")).andThen(d -> d.ladrar( ))); // NO, da error

```

Pero esto produce un ERROR. Sin embargo si podríamos usar consumidores con nombre para ambos:

```

Consumer<Perro> mostrarNombre = d -> System.out.print(d + " ");
Consumer<Perro> ladrado = d -> d.ladrar( );
perros.forEach(mostrarNombre.andThen(ladrado));

```

Trabajando con Predicados

Anteriormente creamos un interfaz `PaseadorPerros` y lo usamos para crear una clase interna y después una expresión lambda:

```
interface PaseadorPerros{
    public boolean pasear(Perro p);
}
```

Este interfaz es un **interfaz funcional**, con un método funcional **pasear()**, así que aunque podemos crear una instancia usando una clase interna:

```
PaseadorPerros dq = new PaseadorPerros ( ) {
    public boolean pasear (Perro p){ return d.getEdad( ) > 9; }
};
```

También nos dimos cuenta de que podíamos ser más concisos usando una expresión lambda:

```
PaseadorPerros dq = d -> d.getEdad( ) > 9; // Reemplaza a la clase interna
```

Si echamos un vistazo a **java.util.function.Predicate**:

```
@FunctionalInterface
public interface Predicate<T> {
    boolean test(T t);
    // métodos static y default aquí
}
```

Predicate es un interfaz funcional, con un método funcional `test()`, que retorna un boolean, igual que `PaseadorPerros`. Podríamos usarlo en su lugar:

```
Perro perro1 = new Perro("Sultan",30,6);
Perro perro2 = new Perro ("Canelo",35,12);
Predicate<Perro> p = d -> d.getEdad( ) > 9;
System.out.println(" ¿Tiene Sultán más de 9 años? " + p.test(perro1));
System.out.println(" ¿Tiene Canelo más de 9 años? " + p.test(perro2));
```

Hay que darse cuenta de que debemos añadir el tipo del parámetro a `Predicate` cuando usamos `Predicate` en lugar de `PaseadorPerros`. `Predicate` es un interfaz funcional genérico, por lo que tenemos que darle algo más de información. El argumento `T` definido en el método `test()` significa que podemos pasar cualquier objeto a `Predicate`. Añadiendo el tipo de parámetro a `Predicate`, este puede conocer el tipo de argumento que puede esperar.

```
public class Perro {
    private String nombre;
    private int edad;
    private int peso;

    Perro(String nombre, int peso, int edad)
    {
```

```

    this. nombre = nombre;
    this. peso = peso;
    this. edad = edad;
}
// getters y setters aquí.
public String getNombre( ) { return this.nombre; }
public void ladrar( ) { System.out.println("Guau!"); }
}

public class TestPerroPredicates {
    public static void main(String[ ] args) {
        ArrayList<Perro> perros = new ArrayList( );
        Perro perro1 = new Perro("Sultan",30,6);
        Perro perro2 = new Perro ("Canelo",35,12);
        Perro perro3 = new Perro("Lucas", 45, 8);
        perros.add(perro1); perros.add(perro2); perros.add(perro3);

        System.out.println("--- Todos los perros---");
        perros.forEach(d -> System.out.println(d));

        System.out.println("--- Perros más jóvenes de 9 ---");
        mostrarPerroIF (perros, (d) -> d.getAEdad( ) < 9);

        System.out.println("--- Perros mayores ó de 9 ---");
        mostrarPerroIF (perros, (d) -> d.getEdad( ) >= 9);
    }
    public static void mostrarPerroIF (ArrayList<Perro> perros, Predicate<Perro> p) {
        for(Perro d : perros) {
            if(p.test(d)) {
                System.out.println(d); }
        }
    }
}

```

Lo que hemos hecho aquí es crear un método **mostrarPerroIF** que toma una lista de perros y un **Predicate**, y prueba cada perro en la lista contra el predicado para ver si el perro debe mostrarse. Entonces podemos usar este método para mostrar todos los perros menores de 9 años con un predicado y mostrar todos los perros de 9 años o más con otro.

- **Métodos default y static de Predicate**

El interfaz **Predicate**, definido dentro del paquete **java.util.function**, tiene algunos métodos predeterminados y un método estático. Los métodos **and()**, **or()** y **negate()** están ahí para que pueda encadenar predicados juntos, al igual que lo hicimos con los consumidores y el método **andThen()**. Esta puede ahorrando tiempo creando nuevos predicados que son combinaciones lógicas de predicados que ya tenemos.

```

Predicate<Perro> edad = d -> d.getEdad( ) == 6;
System.out.println("¿Tiene Canelo 6 años?" + edad.negate( ).test(perro2));

```

Los métodos **or()** y **and()** toman otros predicados, así que para encadenarlos juntos necesitamos dos predicados:

```
Predicate<Perro> nombre = d -> d.getNombre( ).equals("Lucas");
Predicate<Perro> edad = d -> d.getEdad( ) == 6;
Predicate<Perro> nombreYEdad = d -> nombre.and(edad).test(d);
System.out.println("--- Probar nombre y edad de Lucas ---");
System.out.println("¿Se llama Lucas y tiene una edad de 6?" + nombreYEdad.test(perro3));
perro3.setEdad(7);
System.out.println("¿Se llama Lucas y tiene una edad de 6?" + nombreYEdad.test(perro3));
```

Se podría simplificar más escribiendo:

```
Predicate<Perro> nombreYEdad = nombre.and(edad);
```

El método estático en **Predicate**, **isEqual()**, sólo le brinda una forma de probar si un objeto es igual a otro:

```
Predicate<Perro> = Predicate.isEqual(perro3);
System.out.println("Es Lucas el mismo objeto que Canelo?" + p.test(perro2));
System.out.println("Es Lucas el mismo objeto que Sultán?" + p.test(perro1));
```

Una cosa a tener en cuenta sobre **isEqual()** es que este método se define solo en predicados que toman objetos como argumentos.

Junto con **Predicate**, encontramos **BiPredicate**, **DoublePredicate**, **IntPredicate** y **LongPredicate** en **java.util.function**. El método **test()** de **BiPredicate** requiere dos argumentos, mientras que **DoublePredicate**, **IntPredicate** y **LongPredicate** cada uno toma un argumento de tipo primitivo (para evitar el autoboxing).

```
IntPredicate respuesta = i -> i == 42;
System.out.println("¿Es la respuesta 42?" + respuesta.test(42));
```

- **BiPredicate**

BiPredicate es sólo una variación de **Predicate** que permite pasar dos objetos para probar en lugar de uno solo. Digamos que tenemos una **ArrayList** de libros y queremos crear un conjunto de predicados que determinarán si debemos comprar un libro en función de su nombre, su precio o su nombre y precio juntos.

```
List<Book> libros = new ArrayList<>( );
BiPredicate<String, Double> javaComprar = (nombre,precio) -> name.contains("Java");
BiPredicate<String, Double> precioComprar = (nombre,precio) -> price < 55.00;
BiPredicate<String, Double> comprarDefinitivamente = javaComprar.and(precioComprar);

books.forEach(libro -> {
    if (comprarDefinitivamente.test(libro.getNombre( ), libro.getPrecio( ))) {
        System.out.println("Debes comprarte " + libro.getNombre( )
            + " (" + getPrecio( ) + ")");
    }
});
```

Trabajando con Funciones

El propósito del método funcional **apply()** en el interfaz **Function** es tomar un valor y convertirlo en otro. Los dos valores no tienen que ser el mismo tipo, por lo que en la definición de interfaz de función, tenemos dos tipos diferentes parámetros, T (el tipo de argumento de **apply()**) y R (el tipo de argumento valor de retorno):

```
@FunctionalInterface
public interface Function<T,R> {
    R apply (T t);
}
```

Podemos crear una instancia de **Function** para convertir un **Integer** en un **String**:

```
function<Integer,String> respuesta = a -> {
    if(a== 42) return "cuarenta y dos";
    else return "No hay respuesta!";
};
System.out.println(respuesta.apply(42));
System.out.println(respuesta.apply(64));
```

Una **BiFunction** es similar, excepto que el método **apply()** toma dos argumentos y devuelve un valor:

```
Bifunction<String,String,String> apellidos = (primero, segundo) -> primero + " " + segundo;
System.out.println("Nombre y apellido: " + apellidos.apply("Juan", "Valdés"));
```

En este ejemplo, el método **BiFunction apply()** toma dos cadenas y devuelve una cadena, pero podría pasar dos argumentos de diferentes tipos y devolver un valor de un tercer tipo.

- **Funciones en el JDK**

Usemos un **Function** y un **BiFunction** en un ejemplo usando los métodos de **Map** **computeIfAbsent()** y **replaceAll()**:

```
public class Functions {
    public static void main(String[] args) {
        Map<String, String> ganadorAbril = new TreeMap<>( );
        ganadorAbril.put("Abril 2017", "Luis");
        ganadorAbril.put("Abril 2016", "Ana");
        ganadorAbril.put("Abril 2015", "Lorenzo");
        System.out.println("--- lista, antes comprobando Abril 2014 ---");
        ganadorAbril.forEach((k,v) -> System.out.println(k + ": " + v));

        // No hay clave para Abril 2014, así que "Juan Pérez" es añadido al mapa
        ganadorAbril.computeIfAbsent("Abril 2014", (k) -> "Juan Pérez" );

        // La clave Abril 2014 ya existe, por lo que "Juan Gómez" no será añadido
        ganadorAbril.computeIfAbsent("Abril 2014", (k) -> "Juan Gómez" );
    }
}
```

```

ganadorAbril.forEach((k,v) -> System.out.println(k + ": " + v));

// Usamos una BiFunction para reemplazar todos los valores en el mapa con mayúsculas.
ganadorAbril.replaceAll((key, oldValue) -> oldValue.toUpperCase( ));
System.out.println("--- lista después de reemplazar los valores con mayúsculas ---");

ganadorAbril.forEach((k,v) -> System.out.println(k + ": " + v));
}
}

```

- **Más funciones**

Function tiene un par de métodos default y un método static además de su método funcional, `apply( )`: `andThen( )`, `compose( )`, e `identity( )`. `andThen( )` es similar al método de `Consumer` `andThen( )`, aplicando `Functions` en secuencia. `compose( )` es igual excepto que aplica `Functions` en orden inverso. Y el método estático en `Function` es **`identity( )`**, que sólo retorna su argumento:

```

Función<Integer, Integer> id = Function.identity( );
System.out.println(id.apply(42));

```

¿Para qué se usa una función de identidad? Imaginemos un escenario donde se ha definido un método que toma un **Function** como un argumento que cambia un valor en una estructura de datos. Pero en algunos casos, no desea que ese valor cambie. En esos casos, pasamos la función de identidad como una operación "no hacer nada" fácil.

Junto con **Function** y **BiFunction**, como es de esperar, también encontramos **DoubleFunction** (**`apply( )`** toma un doble como argumento y devuelve un objeto), **IntFunction** (**`apply( )`** toma un int como argumento y devuelve un objeto), y **LongFunction** (**`apply( )`** toma un long como argumento y devuelve un objeto). También tenemos **DoubleToIntFunction**, **DoubleToLongFunction**, **IntToDoubleFunction**, **IntToLongFunction**, **LongToDoubleFunction**, **LongToIntFunction**, **ToDoubleFunction**, **ToIntFunction**, **ToLongFunction**, **ToDoubleBiFunction**, **ToIntBiFunction**, y por último pero definitivamente no menos importante, **ToLongBiFunction**.

Trabajando con Operadores

Finalmente, tenemos las variaciones del operador en los interfaces funcionales. Todos los **Operators** son, de hecho, versiones ligeramente modificadas de otros interfaces funcionales. Como ejemplo, nos fijaremos solamente en **UnaryOperator**.

UnaryOperator extiende el interfaz **Function**, por lo que su método funcional es también **`apply( )`**. Sin embargo, a diferencia de **Function**, requiere que el tipo del argumento de **`apply( )`** será el mismo que el tipo del valor de retorno, de forma que **UnaryOperator** se define así:

```

@FunctionalInterface
public interface UnaryOperator<T> extends Function<T,T>

```

La **T, T** en el tipo de los parámetros de **Function** es lo que le indica que el tipo del argumento y el valor de retorno deben ser los mismos. Por eso solo se tiene que especificar un tipo de parámetro para **UnaryOperator**:

```

UnaryOperator<Double> log2 = v -> Math.log(v) / Math.log(2);

```



```
System.out.println(log2.apply(8.0));
```

En este ejemplo, estamos definiendo un **UnaryOperator** Log2 que calcula el logaritmo en base 2 de un valor. El logaritmo en base 2 de 8.0 es 3.0 porque $2 * 2 * 2$ (3 veces) es 8.

Referencias a métodos

Ya sabemos que una expresión lambda es una forma abreviada de escribir una instancia de una clase que implementa una interfaz funcional. Por extraño que parezca, hay algunas circunstancias en las que podemos hacer nuestro código aún más conciso escribiendo una abreviatura para la expresión lambda. Algunas veces, lo único que hace una expresión lambda es llamar a otro método:

```
List<String> arboles = Arrays.asList("pino","cedro","chopo");
arboles.forEach(t -> System.out.println(t));
```

Aquí estamos usando una expresión lambda para tomar un nombre tree, t, y pasárselo a System.out.println(). Esto se puede abreviar más usando una "referencia un método:"

```
List<String> arboles = Arrays.asList("pino","cedro","chopo");
arboles.forEach( System.out::println);
```

¿Dónde fue el argumento t? Sabemos que forEach() toma un **Consumer**, y sabemos que lo está consumiendo son los nombres tree, que son Strings. Y sabemos que System.out.println() toma una cadena. Se puede inferir mucho de esta abreviatura para la expresión lambda. Aquí lo que se infiere es que queremos llamar al método println() de System.out, pasando el objeto String que tenemos a la mano a través de forEach().

Tipos de referencias a métodos

Podemos crear referencias a métodos para nuestros propios métodos:

```
public class MetodoRefs {
    public static void main(String[] args) {
        List<String> arboles = Arrays.asList("pino","cedro","chopo");
        arboles.forEach(t -> System.out.println(t)); // Escribirla con una lambda
        arboles.forEach(System.out::println); // Escribirla con una referencia a método
        arboles.forEach(MetodoRefs::printTreeStatic); // Imprimirla con nuestra propia
                                                         // referencia estática a método
    }
    public static void printTreeStatic(String t) {
        System.out.println("Nombre del árbol: " + t);
    }
}
```

Primero, los nombres de los tree se imprimen usando una expresión lambda; después los nombres se imprimen utilizando una **referencia a método de instancia** a `System.out.println( )` y finalmente una cadena más larga usando una **referencia a método estático** a nuestro propio método estático, `printTreeStatic( )`.

La referencia del método a `System.out.println( )` es una **referencia a método de instancia** ", es decir, una referencia al método de instancia `println( )` de `System.out`.

Escribir nuestro propio Interfaz Funcional

Imaginemos, por ejemplo, que deseamos una interfaz con un método `test( )` que toma tres objetos y devuelve un valor booleano:

```
@FunctionalInterface
interface TriPredicate<T, U, V> {
    boolean test (T t, U u, V v) ;
}
```

Estamos especificando una interfaz con tres parámetros de tipo: T, U y V. El método `test( )` toma tres tipos de objetos para probar, y todos pueden ser de diferentes tipos o todos iguales. Ahora vamos a escribir una expresión lambda de esto:

```
public void triPredicate( ) {
    TriPredicate<String, Integer, Integer> theTest = (s,n,w) -> {
        if (s.equals("No hay cuchara") && n > 2 && w < n) {
            return true;
        } else {
            return false;
        }
    };
    System.out.println("¿Pasa el test?" + theTest.test("Sigue al conejo blanco", 2 ,3 ));
    System.out.println("¿Pasa el test?" + theTest.test("No hay cuchara", 101 ,3 ));
}
```

Revisión de los interfaces funcionales

Interface	Functional Method	Description
Supplier<T>	<code>T get ()</code>	Suppliers supply results. The <code>Supplier</code> 's functional method <code>get ()</code> takes no arguments and supplies a generic object result (T).
Consumer<T>	<code>void accept(T t)</code>	Consumers consume values. The <code>Consumer</code> 's functional method <code>accept ()</code> takes a generic object argument (T) and returns no value.
Predicate<T>	<code>boolean test(T t)</code>	Predicates test things (and do logical operations). The <code>Predicate</code> 's functional method <code>test ()</code> takes a generic object argument (T) and returns a boolean.
Function<T,R>	<code>R apply(T t)</code>	Functions are generic functional interfaces. The functional method <code>apply ()</code> takes a generic object argument (T) and returns a generic object (R).
IntSupplier	<code>int getAsInt ()</code>	Supplies int values (to avoid autoboxing). Note the difference in the functional method name (<code>getAsInt ()</code>) rather than <code>get ()</code> , as for <code>Supplier</code> . This pattern is used across the various primitive versions of the functional interfaces.
IntConsumer	<code>void accept(int value)</code>	Consumes int values.
IntPredicate	<code>boolean test(int value)</code>	Tests int values.
IntFunction<R>	<code>R apply(int value)</code>	The <code>IntFunction</code> 's functional method <code>apply ()</code> takes an int argument and returns a generic object result (R).
BiConsumer<T,U>	<code>void accept(T t, U u)</code>	Consumes two values.
BiPredicate<T,U>	<code>boolean test(T t, U u)</code>	The <code>BiPredicate</code> 's functional method <code>test ()</code> takes two generic arguments (T, U).
BiFunction<T,U,R>	<code>R apply(T t, U u)</code>	The <code>BiFunction</code> 's functional method <code>apply ()</code> takes two generic arguments (T, U) and returns a generic object result (R).
UnaryOperator<T>	<code>T apply(T t)</code>	The <code>UnaryOperator</code> 's functional method <code>apply ()</code> takes one generic operator (T) and returns a value of the same type (T).

Sabemos que las interfaces funcionales son solo interfaces con un método abstracto: el método funcional. Las interfaces de la Tabla se han agregado al JDK para que nos sea más fácil escribir código, pero, no hay nada particularmente especial en los interfaces funcionales más allá de su uso en el JDK.

Ya hay algunas interfaces funcionales en el JDK, no listadas en **java.util.function**. Ya hemos visto uno de estos: **Comparator**: con su método funcional, **compare()**. Otro es **Comparable**, que se implementa mediante varios tipos (como String, LocalDateTime, etc...) para ordenar. Un tercer ejemplo es **Runnable**, que tiene un método funcional: **run()**. Entonces, dónde se podría haber implementado un Runnable como una clase interna:

```
Runnable r = new Runnable( ) {  
    @Override  
    public void run( ) {  
        System.out.println("Hazlo");  
    }  
};
```

Ahora podemos usar, en vez de esto, una expresión lambda:

```
Runnable r = ( ) -> System.out.println("Hazlo");
```